

Examen Parcial de IA

(5 de Abril de 2024)

Duración: 90 min.

1. (6 puntos) El mago Rincewind se encuentra en unas ruinas antiguas con un mecanismo similar a un circuito. El espacio del mecanismo está compuesto por C columnas verticales, cada una de H altura máxima. Hay un conjunto de piezas diferentes en la sala para componer el circuito dentro de este espacio. Cada pieza puede ocupar una única columna, y tiene una altura h_p . Una pieza se puede colocar en el suelo de la columna, o encima de otra pieza ya colocada, pero nunca flotando en el aire. Además, las piezas tienen por cada lado una configuración de *thaumaje* o flujo mágico. Por ejemplo, una pieza de altura 3 puede tener *thaumaje* (de abajo a arriba) *azul-verde-amarillo* por un lado, y *azul-azul-azul* por el otro lado. El *thaumaje* es importante, porque dos piezas en columnas colindantes que comparten una altura con el mismo color están conectadas.

Además, hay dos puertos, uno en cada lado de la terminación del circuito (izquierda y derecha), a alturas diferentes H_i y H_d , con *thaumaje* diferente. El objetivo del mecanismo es conectar ambos puertos mediante las piezas que hay en la sala, para lo cual debe existir un camino de piezas conectadas de forma que ambos extremos estén también conectados a los puertos (ej. que la pieza más a la izquierda del camino tenga el *thaumaje* correcto a la altura H_i).

Para resolver el mecanismo es necesario que no haya piezas flotando, que los puertos estén conectados como se ha descrito, que no haya piezas superimpuestas y que no haya piezas que superen la altura máxima del mecanismo H . Además, cada región colindante entre dos piezas que tenga *thaumaje* diferente implica una pérdida de flujo importante, y se debe minimizar. Por último, Rincewind es un vago, y quiere minimizar el número de piezas pesadas que le toca poner en el mecanismo. El peso de una pieza está descrito como W_p .

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (solución inicial, operadores, función heurística, ...). El objetivo es comentar la solución que se propone respecto a si es correcta, es eficiente, o es mejor o peor respecto a otras alternativas posibles para el mismo algoritmo. Justifica tu respuesta.

- a) Se plantea solucionarlo mediante Hill-Climbing. Empezamos desde la solución vacía. Como operadores, tenemos un operador `colocar(P,C)` que coloca la pieza P en la columna C encima de las que ya haya y `swap(P1,P2)` que intercambia dos piezas colocadas en el mecanismo. Como heurística se propone el número total de *thaumajes* incompatibles en la solución más el número de piezas conectadas al puerto izquierdo y derecho al cuadrado (por separado), más el peso total de las piezas asignadas a la solución actual:

$$\sum_{c \in \{1..C-1\}} \sum_{h \in H} th_incompat(c, c+1, h) + conectadas_izq()^2 + conectadas_der()^2 + \sum_{p \in P} W_p * colocada(P) \quad (1)$$

Se analiza la solución de forma esquemática y separada por bloques en función de la dificultad de análisis. Entre paréntesis se añade clarificación que no era necesaria para conseguir el puntaje.

Puntos de análisis sencillo (se espera este análisis como mínimo):

- (0.2) HC no es un buen algoritmo para un problema en que encontrar una solución es difícil (conectar puertos no 'parece' trivial). Puede funcionar bien con buenos operadores y heurísticos.
- (0.2) La solución inicial no es solución. Es difícil empezar con una solución. Sería mejor empezar con una asignación de piezas aleatoria para poder reiniciar el algoritmo varias veces.
- (0.2) Los operadores, empezando desde la solución vacía, permiten explorar todo el espacio de soluciones.
- (0.2 extra) Un operador de quitar pieza puede ser útil, especialmente si cambiamos la solución inicial.
- (0.2) Los operadores no comprueban que una columna no sobrepase la altura H .
- (0.2) La ramificación de los operadores es $O(P * C)$ y $O(P * (P - 1)/2)$ respectivamente. (Quitar sería $O(P)$).
- (0.2) El heurístico es incorrecto, tiene cantidades a minimizar: pesos y incompatibilidades; y cantidades (aparentemente*) a maximizar: piezas conectadas por lado. Tampoco tienen las mismas unidades. Sería mejor cambiar el signo para tener la misma dirección de optimización y añadir ponderadores.

Puntos de análisis complejo (puntos más complejos de analizar pero que son necesarios para una comprensión holística de la propuesta):

- (0.4) La solución inicial propuesta, junto a los operadores y un buen heurístico, es un óptimo local con alta probabilidad: añadir una pieza empeora el heurístico. (El único caso en que no se da es cuando uno de los dos puertos esté muy bajo y pueda ser conectado con la primera pieza.) Alternativamente, se debería comentar que el heurístico tenderá a eliminar piezas de la solución para ir hacia la solución vacía con alta probabilidad (eg. siempre que quitar la pieza no reduzca el número de piezas conectadas a un puerto).
- (0.4) El heurístico **no** busca lo que se pide en el enunciado, e ignora la conexión entre puertos (no se penaliza que no estén conectados). Las cantidades cuadradas buscan maximizar algo que no se pide: el número de piezas conectadas. (Por ejemplo, tener 5 piezas conectadas en las dos columnas de más a la izquierda (patrón zig-zag) es mucho mejor que tener 4 piezas conectadas que acerquen a los dos puertos.)

Puntos de análisis extra (puntos que no se espera que el alumno analice pero que se han valorado positivamente si se ha hecho):

- (0.4 extra) Una manera de resolver el problema del heurístico es 'penalizar' la distancia mínima entre piezas conectadas al puerto izquierdo y al derecho.
- (0.4 extra) Esto aplica a todos los apartados, pero se ha puntuado en el 1a: siendo piezas de un mecanismo tridimensional, es posible que el modelado esté ignorando la posibilidad de girar piezas sobre sí mismas, cambiando orientaciones y configuraciones de alturas. (Comentar este hecho puede ser interesante, pero son puntos extra porque va más allá de la descripción del enunciado, y puede que no sea posible girar piezas dependiendo de factores desconocidos.)

b) Se plantea resolverlo mediante algoritmos genéticos. La representación consiste en una ristra de $P * (\lceil \log_2 C \rceil + N)$ bits. Cada uno de los P bloques representa en binario la id de la columna donde está el pieza, y un número en binario que 'desempata' piezas de la misma columna y marca el orden en alturas (por ejemplo: "010(C)-11(N) | 010(C)-00(N)" implica que ambas piezas van a la columna 3 (010) y que la pieza 1 está encima de la pieza 2 (porque 3 es más grande que 0). En caso de empate, se ordenan en orden de id de pieza. Como heurística usamos la siguiente:

$$conectadas_a_puerto() - \lambda * \sum_{p \in P} W_p * colocada(P) - \gamma * \sum_{c \in \{1..C-1\}} \sum_{h \in H} th_incompat(c, c+1, h) \quad (2)$$

Puntos de análisis sencillo:

- (0.3) Genéticos sufre del mismo problema que HC: no garantiza estar en el espacio de soluciones si este es difícil de encontrar, pero cabe la posibilidad. La robustez de la búsqueda hace más sencillo tener un buen heurístico.
- (0.2) La representación permite representar cualquier solución*...
- (0.4) excepto porque no permite tener piezas no colocadas. Para ello basta con añadir una ID de columna (por ejemplo, 0) que indique que la pieza no está colocada. Esto permite representar cualquier solución...
- (0.2 extra) siempre y cuando 2^N sea suficientemente grande para poder tener cualquier orden en cualquier columna (por ejemplo, si caben como máximo 8 paquetes en una columna, N debería valer 3 para poder representar el orden opuesto al de ids de piezas).
- (0.3) La representación permite asignar piezas a columnas inexistentes si C es diferente de una potencia de 2.

Puntos de análisis complejo:

- (0.4) La heurística no penaliza que haya piezas que superen la altura H .
- (0.4) La heurística pide maximizar las piezas conectadas a puerto, pero esto no es lo que se pide. La heurística **no** penaliza tener los puertos desconectados (ni ninguna aproximación a ello).
- (0.3) La representación permite asignar piezas a columnas inexistentes si C es diferente de una potencia de 2. Falta una penalización en el heurístico.

Puntos de análisis extra:

- (0.4 extra) Las columnas fantasma y la no-asignación se pueden resolver a la par: si una pieza se asigna a una columna inexistente, se considera no-asignación. (Esta opción es mejor a la de tener un id aparte ya que parece 'probable' tener piezas no-asignadas, e incrementar la probabilidad de que esto ocurra de forma natural es bueno).

c) Se plantea resolverlo mediante CSP. Tenemos P variables cuyos dominios son las C columnas más la no-asignación. Tenemos también una variable W de peso total, y una variable I que cuenta las incompatibilidades de flujo. Por último, tenemos una variable O que suma ambas, y se pide minimizar. Como restricciones, tenemos P restricciones de preprocesado que impide poner piezas que se sobrepasen de la altura H , una n -aria entre P y la variable I que hace que I valga el número de incompatibilidades en la asignación actual, una restricción similar para W y una restricción ternaria entre W , I y O que hace que la última valga la suma de las dos anteriores. Por último, tenemos una restricción N -aria entre las P variables que comprueba que ambos puertos están conectados a través de las piezas colocadas.

Puntos de análisis sencillo:

- (0.2) Constraint Programming es una buena opción para cuando se busca cumplir unas restricciones como la conexión entre puertos, y además permite minimizar el resto de criterios.
- (0.6) Las variables no determinan una solución, ya que no sabemos el orden/altura de las piezas en una columna.
- (0.4) La variable objetivo suma unidades diferentes, debería tener ponderadores.
- (0.4) Las constraints propuestas son correctas (si se resuelve la altura).

- (0.4) Las constraints propuestas son extremadamente ineficientes por ser n-arias. No es posible simplificar ninguna de ellas.

Puntos de análisis extra:

- (0.4 extra) Puede proponerse una representación alternativa que permite reducir la complejidad de restricciones. Una opción puede ser: tener K variables por columna (cada una representa una pieza y están en orden) con posibilidad de no-asignación. Esto simplifica las restricciones de altura (a K-arias), de conexión/incompatibilidad (a 2K-arias). Se han aceptado varias propuestas siempre que no tuvieran los problemas mencionados anteriormente.

2. (4 puntos) El área de Parques y Jardines del Ayuntamiento de Barcelona quiere modernizar su flota de mini-furgonetas con una nueva, experimental, impulsada por hidrógeno, que tiene la ventaja de ser altamente ecológica y fácil de mantener pero que tiene una autonomía muy limitada. Para resolver el problema de la autonomía se han colocado 3 surtidores de hidrógeno por diferentes partes de la ciudad para que la mini-furgoneta pueda repostar.

Queremos planificar el recorrido diario de esta mini-furgoneta por los diferentes parques de la ciudad de forma que pueda pasar por todos los parques una vez, tardando el mínimo tiempo posible y pasando por uno de los surtidores cada vez que se le esté acabando el hidrógeno. Como datos para esta planificación disponemos de una tabla que nos indica las posiciones de todos los parques y jardines a visitar y de los tres surtidores, así como la distancia entre cada uno de estos puntos en la ciudad. Sabemos también que la mini-furgoneta experimental tiene un tanque de n litros de hidrógeno y que su consumo es de x litros de hidrógeno por kilómetro. El punto inicial del recorrido diario es el garaje de furgonetas con el tanque lleno de hidrógeno y el punto final es el mismo garaje de furgonetas (donde rellenan el tanque y revisan la mini-furgoneta para que esté perfecta el día siguiente). Los puntos intermedios del recorrido solo pueden ser parques o surtidores.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (estado o solución inicial, operadores, función heurística,...). Para cada una de ellas se ha de comentar los diferentes elementos de la propuesta, analizando si la técnica escogida es adecuada para este problema, si cada uno de sus elementos son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

- a) Queremos utilizar A^* para minimizar el número de kilómetros recorridos por la furgoneta y reducir el número de repostajes al mínimo. Partimos de un estado inicial vacío. El operador propuesto es pasar del punto actual a otro punto. Utilizaremos como función de coste los litros de hidrógeno consumidos (se calcula a partir de los kilómetros recorridos del punto actual al escogido, excepto cuando el punto escogido es un surtidor, ya que en ese caso el coste son los litros de hidrógeno consumidos del punto actual al surtidor escogido menos los litros de hidrógeno cargados en el repostaje). La función heurística vale infinito si la mini-furgoneta no tiene suficiente combustible para ir del punto actual a un surtidor, y en caso contrario estima los litros de hidrógeno por consumir calculando la suma de las distancias de los puntos por recorrer al punto actual y multiplicando el resultado por los x litros de hidrógeno por kilómetro que consume.

Puntos de análisis sencillo:

- (0.2) A^* puede ser adecuado ya que se nos pide el mínimo global, siempre y cuando exista una buena heurística para guiarlo. Seguramente no sea el caso (el hecho de ser similar a TSP debería daros una pista).
- (0.2) El estado inicial vacío es correcto para A^*
- (0.2) El operador permite alcanzar todo el espacio de soluciones empezando desde la solución vacía.
- (0.2) La ramificación del operador es $O(P + S)$

Puntos de análisis complejo:

- (0.4) El coste del operador no es lo que se pide minimizar, ya que hace que g calcule el número de litros de combustible que faltan en el depósito en un instante (dado que se reduce al repostar). (Decir que el coste del operador es correcto porque el combustible es proporcional a la distancia se ha considerado correcto siempre y cuando se comenta el punto posterior).
- (0.4) El coste del operador puede ser (y probablemente sea) negativo al repostar, lo cual es un error grave en A^* . (El algoritmo entra en bucle infinito yendo a repostar).
- (0.2) La heurística 'infinita' al quedarnos sin combustible no es 'incorrecta' (pero no es necesaria: al abrir estados donde no se alcanza un surtidor sencillamente no generan estados sucesores y se cierran).
- (0.2) La heurística es claramente inadmisibile. Un contraejemplo de admisibilidad: supongamos N puntos a equidistancia de 1 entre sí, y el punto actual ubicado a distancia 100. La heurística se valora como $100 * N$, mientras que la heurística h^* se valora como $100 + N - 1$.

- b) Queremos utilizar Hill Climbing para minimizar el número de kilómetros recorridos por la mini-furgoneta y reducir el número de repostajes al mínimo. Partimos de una solución inicial en la que se genera aleatoriamente un orden de visita de los parques y se intercala un paso por un surtidor de hidrógeno al azar entre parque y parque. Se dispone de dos operadores de modificación de la solución: uno para eliminar del camino un paso por un surtidor, y otro para modificar el orden en el que visitamos alguno de los puntos del camino (ya sea parque o surtidor), desplazándolo adelante o atrás en el orden una cantidad n de posiciones. La función heurística es la longitud del camino recorrido + 2 kilómetros extra de penalización por cada paso por un surtidor.

El problema está sobre 2.2 puntos (más allá de los puntos extra).

Puntos de análisis sencillo:

- (0.2) HC puede ser un buen algoritmo para minimizar, ya que aunque se nos pide el óptimo global, en un problema de estas características puede no ser posible. (También se acepta argumentos diciendo que HC no es buen algoritmo ya que no garantiza optimalidad global.)

- (0.3) La solución inicial es aparentemente solución*(mirar análisis extra) pero tiene una calidad pésima.
- (0.2) Los operadores permiten (a priori) explorar todo el espacio de soluciones (que nos interesa*, mirar análisis complejo)
- (0.2) La ramificación de los operadores es $O(P)$ (hay como máximo un número P de surtidores en la ruta) y $O(P^2)$.
- (0.4) Los operadores ignoran la autonomía. Deberían tener condición de aplicabilidad (para no quitar surtidores necesarios, o cambiar la ruta de forma que dejen el vehículo sin combustible). (Propuestas de penalizarlo en la heurística se han evaluado en función de si el cambio propuesto asegura estar en el espacio de soluciones al final).
- (0.4) El heurístico es correcto **excepto** por la penalización de 2 kilómetros extra: no es algo que se pida y puede implicar escoger soluciones menos óptimas en las que se visitan surtidores más veces (por ejemplo, repostar en un surtidor que queda entre dos parques dos veces sería peor que repostar una vez yendo a un sitio a 2 km de distancia). La penalización es sencillamente innecesaria.

Puntos de análisis complejo:

- (0.2) Los operadores irreversiblemente quitan surtidores de la ruta. Esto puede causar que uno de los surtidores sea completamente eliminado de la ruta y no se pueda recuperar.
- (0.3) Un operador como 'añadir surtidor' parece buena idea para arreglar el punto anterior. Sin embargo, será inútil para resolver el problema anterior ya que siempre aumenta la heurística. Un operador de 'intercambiar' un surtidor por otro puede resolver el problema.

Puntos de análisis extra:

- (0.2 extra) La solución inicial puede no ser solución si hay distancias entre un surtidor y un parque que exceda la autonomía (por ejemplo, un recorrido de parque A a surtidor más cercano SA, y de forma aleatoria se escoja el parque más lejano a SA). Es un 'edge case' pero se ha valorado el denotarlo.
- (0.2 extra) Mejor que un operador para intercambiar surtidores, puede ser mejor un operador de 'intercambiar y mover', que elimina un surtidor de la ruta y coloca un surtidor (potencialmente uno de los otros dos) en otra posición de la ruta para asegurar autonomía.

Las notas se publicarán el día **26 de abril**.